AI Security & Compliance Assistant

Product & Technical Documentation

Prepared for: Security, Compliance & Engineering Stakeholders

Document type: Product overview, architecture, and governance reference

Version: 1.0 (Draft — informed by internal Chrome extension research brief)

Table of Contents

1. Overview — What This Extension Does 3
2. Why We Need This Extension 3
3. Core Features 4
4. Technology Stack 4
5. Architecture 4
6. How We Will Manage This 5
7. Pros & Cons of Building on the Chrome Extension Platform 5
8. Lessons From Extension History — Five Scenarios 6
9. How We Built This 7
10. Security, Privacy & Compliance Commitments 8
11. Roadmap 8
12. Sources & References 8

1. Overview — What This Extension Does

AI Security & Compliance Assistant is a Manifest V3 Chrome extension that gives employees real-time, in-browser guidance on security and compliance decisions — the moment those decisions happen, not after the fact in a training module.

It sits quietly in the browser and activates in three situations:

- An employee is about to do something the policy documents flag as risky (upload a file to an unapproved service, paste sensitive text into an external tool, visit a high-risk domain).
- An employee has a compliance or security question and wants a direct, policy-grounded answer instead of searching a wiki or opening a ticket.
- The organization needs visibility into where policy ambiguity or risky behavior is concentrated, without collecting content itself.

It is deliberately not a general-purpose monitoring tool. It is scoped, from day one, to a small number of high-value features rather than the widest possible feature set — a direct response to the finding (see Section 8) that broad permissions are themselves a security liability, independent of intent.

2. Why We Need This Extension

The problem: Most accidental policy violations and data leakage don't happen because employees don't know the rules — they happen because the rules aren't visible at the moment a risky action is taken. By the time annual training or a policy PDF would have helped, the file is already uploaded or the paste has already happened.

The business case:

Driver	Current state without the extension	Expected change with the extension
Accidental data leakage	Discovered after the fact, via DLP alerts or incident review	Flagged before the action completes
Policy accessibility	Buried in documents employees rarely open	Answered inline, grounded in the actual policy text
Compliance team load	Repeated, low-complexity policy questions routed to humans	Deflected to a self-serve, policy-grounded assistant
Security awareness	Point-in-time (annual training)	Continuous, contextual, just-in-time
Audit posture	Reactive — reconstructed after incidents	Proactive — decision points are logged and improvable

What it does not do: replace security training, replace DLP/CASB tooling, or serve as a monitoring/surveillance system. It is a decision-support layer, not an enforcement or surveillance layer — a distinction that matters for both employee trust and the privacy commitments in Section 10.

3. Core Features

Feature	What it does
Policy Copilot	Answers employee questions about company policy in plain language, grounded strictly in the attached governance documents — never inventing policy that isn't there.
Context-Aware Security Warnings	Just-in-time nudges when an employee is about to take an action a policy flags as risky (e.g., pasting into an unapproved AI tool).
Secure Decision Assistant	When an employee is unsure whether an action is allowed, walks them through the relevant policy logic instead of giving a bare yes/no.
Compliance Coach	Explains why something is a violation in plain terms, tying the explanation back to the specific policy clause.
Secure Browsing / Domain Risk Control	A declarativeNetRequest-based filtering engine that blocks or warns on known high-risk domains, using locally compiled and indexed rule sets — not remote lookups — for speed and privacy. Rules are stored via chrome.storage.local, synced across a user's devices via chrome.storage.sync, manageable from a lightweight options page (with dark mode and offline support), and enforced without any dependency on network availability.
Intelligent Security Nudges	Lightweight, low-friction reminders (not blocking pop-ups) designed around behavioral-science principles rather than compliance-by-interruption.

4. Technology Stack

Layer	Technology	Notes
UI (popup, options page, side panel)	React, TypeScript, HTML, CSS	Component-based, typed for maintainability
Extension runtime	Chrome Extension Manifest V3	Service worker instead of persistent background page
Content detection	Content scripts (isolated world)	Scoped to specific page contexts, not <all_urls> by default
Filtering engine	declarativeNetRequest API	Chosen specifically over the deprecated blocking webRequest API (see Scenario 4, Section 8)
Local storage	chrome.storage.local	Rule sets and configuration kept on-device by default
Cross-device sync	chrome.storage.sync	Only non-sensitive configuration is synced
AI integration	Enterprise-approved AI/LLM service	Called only for policy Q&A and risk explanation; grounded in attached policy documents with guardrails against hallucinated policy claims
Enterprise deployment	Google Workspace Admin Console	Force-installed to managed devices; not distributed via the public Chrome Web Store

5. Architecture

- Service worker — orchestrates AI calls, rule-set compilation, and cross-component messaging. Ephemeral by design (Manifest V3 constraint), so no state is assumed to persist between wake cycles.
- Content scripts — detect risky in-page actions (uploads, paste events into known external tools) and relay minimal signal (not raw content) to the service worker for evaluation.
- declarativeNetRequest rule engine — domain/category rules compiled at build or update time, indexed for fast lookup, enforced entirely client-side.

- Options page — human-manageable rule configuration, category toggles, sync settings.
- AI layer — receives only the minimal context needed to answer a question or evaluate a risk; policy documents are the grounding source, with explicit guardrails against answering outside that grounding (see Section 10).

Design principles applied throughout: **Zero Trust**, **Secure by Design**, **Privacy by Design**, and **Principle of Least Privilege** — each permission and API is justified against a specific feature, not requested speculatively.

6. How We Will Manage This

An internal research brief on Chrome extension history (Section 8) makes one point centrally: the extensions that get abused or become long-term liabilities are the ones without a clear, accountable owner and a permission footprint that's allowed to drift. Management of this extension is built around closing exactly that gap:

- Named ownership. A joint Security + Compliance sponsor owns the extension's roadmap and is accountable for it — there is no scenario where "the extension" exists without a named internal owner, unlike a public extension that can be sold or abandoned.
- Permission governance. Each permission in the manifest is mapped to a specific, currently-shipping feature. Any new permission request requires sign-off before merge. Quarterly audits check the live manifest against this mapping — directly addressing the least-privilege failures documented in Section 8, Scenario 5.
- Controlled distribution. Deployed via the Google Workspace Admin Console to managed devices only — not the public Chrome Web Store — which removes the acquisition-and-resale risk described in Scenario 3 entirely, since there is no independent developer who could sell it.
- Change control. All updates go through internal code review and staged rollout, with a changelog and the ability to force-disable via the admin console if an issue is found post-release.
- Platform-change monitoring. Someone on the team explicitly tracks the Chromium/Manifest roadmap. Manifest V2→V3 is treated as a precedent, not a one-off — the team maintains a documented contingency plan for what happens if a currently-used API (e.g., declarativeNetRequest rule caps) is tightened.
- Data governance. Data handling is reviewed against the attached governance documents and, where those are silent, against named industry frameworks — with every requirement labeled as policy-derived or best-practice, never blended.

7. Pros & Cons of Building on the Chrome Extension Platform

Pros	Cons
Technically low-friction to build and iterate — verifiable by anyone who tries it.	Google has already unilaterally and materially restricted extension capability once, at scale (Manifest V2→V3), and the pattern has historical precedent beyond Chrome.
Extensions get a level of real-time, in-context browser access that no other software category gets — exactly what "just-in-time" guidance needs.	Broad permission requests correlate with both a larger attack surface and long-term erosion of user trust, even when the intent is benign.
For enterprise use, deployment via Workspace Admin Console sidesteps public Chrome Web Store review, ranking, and resale risk entirely.	The extension is built on a platform whose rules the organization does not control — a future API restriction can force a redesign on Google's timeline, not ours.

Pros	Cons
Cross-device configuration sync is built into the platform (chrome.storage.sync).	Publicly, extensions are rarely sustainable as standalone products — for us this reframes as an internal question: who owns and funds this indefinitely, not "how does it monetize."
No separate download or install friction for employees — it lives where the risk actually occurs.	Historically, trusted extensions with a real user base have been acquired and converted into adware once ownership changed hands — a risk we structurally avoid via internal-only ownership, but one worth institutional awareness of.

8. Lessons From Extension History — Five Scenarios

These five scenarios are drawn from an internal research brief analyzing the history and risk profile of the Chrome/browser extension model. Each is paired with what we're doing differently in this extension as a direct result.

#	Scenario (what happened)	Lesson	What we implement
1	Internet Explorer Browser Helper Objects (1997–2009). BHOs gave third-party add-ons unrestricted DOM access with no review process. This was quickly exploited for ad-injecting toolbars, keyloggers, and spyware — and Microsoft didn't add a user-facing way to review or disable them until nearly a decade later.	Unreviewed, unrestricted access to page content becomes an abuse vector, almost regardless of original intent.	We never request unrestricted, always-on page access. Content scripts are scoped to specific contexts, every capability maps to a named feature (Section 6), and changes go through internal review before shipping — there is no "silent" access path.
2	Firefox XUL/XPCOM → WebExtensions (2015–2017). Firefox's legacy extension model gave deep, unrestricted access to browser internals. Mozilla deprecated it for security and architectural reasons; thousands of extensions broke, and some functionality could never be replicated under the new, more restricted model.	Building on a platform's deepest, most powerful (and least sanctioned) hooks is a future liability — deep access gets revoked, not preserved.	We build exclusively on current, documented, stable Manifest V3 APIs. We deliberately avoid the deprecated blocking webRequest model (see Scenario 4) and design each feature as a self-contained module, so a future platform restriction can force a redesign of one feature, not the whole extension.
3	Trusted extensions bought and converted to adware (2014–2018). "Add to Feedly," "Tweet This Page," "Particle" (formerly YouTube+), and "Stylish" were all sold by their original developers and quickly turned into ad-injection or browsing-history-logging tools once ownership changed.	A trusted install base is valuable precisely because it's trusted — and that trust can be resold and weaponized the moment ownership changes.	This risk is structurally avoided rather than mitigated: the extension is internally owned, distributed only via the Google Workspace Admin Console to managed devices, and is not — and cannot be — an independently sellable asset. Named internal ownership (Section 6) removes the "no accountable owner" condition this scenario depends on.
4	Manifest V3 restricts ad-blocker capability (2019–2022). Google restricted the blocking form of the webRequest API in favor of declarativeNetRequest, with a hard rule cap well below what many ad-blocker filter lists required — materially reducing capability for that	Relying on the most powerful available API, rather than the platform's sanctioned, forward-compatible one, creates fragility exactly where you can least afford it.	Our Secure Browsing / Domain Risk Control feature is built on declarativeNetRequest from day one — not the deprecated blocking webRequest API — with rule sets compiled and indexed to stay within Chrome's static rule limits, and monitoring in place so we notice before we approach any cap.

#	Scenario (what happened)	Lesson	What we implement
	category, a change the EFF publicly opposed.		
5	Peer-reviewed research on permission overreach (2016–2025). MIT researchers found Chrome's permission model fails to convey real capability to users. A University of Wisconsin–Madison study found roughly 12.5% of studied extensions could access plaintext passwords in page source. A 2022 Chalmers University study found only 39.8% of analyzed extensions complied with least privilege. A 2025 industry audit found 86% of top extensions request high-risk permissions on install.	Broad permission requests are themselves a measurable security and trust problem — independent of whether the extension is ever actually misused.	Every permission in our manifest is justified against a specific shipping feature (Section 6), audited quarterly against that mapping, and AI/content processing is designed to work with the minimum data footprint necessary — favoring local evaluation over sending page content off-device wherever the feature allows it.

The thread connecting all five: every historical failure traces back to trust granted without a matching accountability structure — no reviewer, no forward-compatible API choice, no accountable owner, or no permission discipline. Our management model in Section 6 is built to make sure this extension never meets any of those conditions.

9. How We Built This

- Started from governance documents, not features. The company's Code of Conduct, Security Policy, Acceptable Use Policy, and Privacy Policy were treated as the primary source of truth. Where no formal Data Classification Policy existed, sensitivity requirements were inferred and explicitly labeled as inferred, never presented as stated policy.
- Treated platform history as a design input, not background reading. Before finalizing the feature set, we reviewed an independent research brief on the history and risk pattern of the Chrome extension model (Section 8) and used its five scenarios as concrete design constraints rather than general awareness.
- Scoped down deliberately. Rather than building the widest possible feature set, we prioritized a small number of high-value features (Section 3), directly responding to the finding that permission breadth is itself a risk (Scenario 5).
- Cross-functional review. Security, Compliance, Engineering, and UX reviewed the design together before build, mapping each requirement to a specific feature (Section 6's permission-governance model exists because of this step).
- Phased delivery. Built as a minimum viable product first (Policy Copilot + Secure Browsing Control), with AI-driven and analytics features layered in later phases (Section 11) rather than attempted all at once.
- Enterprise-only distribution path chosen up front. The decision to deploy via the Google Workspace Admin Console — rather than the public Chrome Web Store — was made specifically to remove the acquisition/resale risk in Scenario 3 as a category, not just mitigate it.

10. Security, Privacy & Compliance Commitments

- Data never collected: raw page content, keystrokes, plaintext credentials, or full browsing history are never captured or transmitted.
- Data that may be processed: minimal contextual signals needed to evaluate a specific risk (e.g., destination domain category, whether a paste event targeted a known external AI tool) — evaluated locally wherever possible.
- AI grounding & guardrails: the AI layer answers policy questions strictly from the attached governance documents; it is designed to decline or flag uncertainty rather than infer or invent policy that isn't explicitly present.
- Retention: configuration and rule data persist locally/synced for functionality; no interaction content is retained beyond what's needed to render a response.
- Requirement labeling: every control in this document is traceable to either an explicit policy requirement or a named industry best practice (e.g., NIST 800-53, OWASP ASVS) — the two are never presented as equivalent.

11. Roadmap

Phase	Focus
Phase 1	Basic Security Assistant — Policy Copilot, Secure Browsing / Domain Risk Control
Phase 2	Compliance Engine — policy validation, violation detection logic
Phase 3	AI Copilot expansion — Secure Decision Assistant, Compliance Coach
Phase 4	Enterprise Analytics — aggregated, non-identifying risk trend reporting
Phase 5	Predictive Security Intelligence — proactive risk surfacing based on aggregate patterns

12. Sources & References

Historical and empirical claims in Section 8 are drawn from an internal research brief citing the following independently verifiable sources:

- Wikipedia, “Browser Helper Object”; Malwarebytes Labs, “Browser Hijack Objects (BHOs)”; eWeek, “Enough with the Browser Toolbars Already” (2009)
- LWN.net, “Deprecating XUL for WebExtensions” (2015); Mozilla Add-ons Blog, WebExtensions FAQ
- PCWorld/IDG, “Spammers buy Chrome extensions and turn them into adware” (2014); BleepingComputer, “‘Particle’ Chrome Extension Sold to New Dev” (2017); The Daily Swig, Stylish extension coverage; Infosecurity Magazine, Honey developer statement
- LWN.net, “Content blockers and Chrome's Manifest V3” (2021); EFF Deeplinks, “Manifest V3 is Deceitful and Threatening” (2021)
- MIT 6.857, “Security Analysis of Chrome Extensions” (2016); University of Wisconsin–Madison research (via AdGuard); Chalmers University of Technology, least-privilege compliance study (2022); Techlicious, citing Cybernews (2025)

This document should be reviewed alongside the attached governance documents and updated whenever the Chromium Manifest roadmap changes materially.